

Java Singleton Patterns - Cheat Sheet

Singleton is the “exactly one instance” pattern. One class, one instance, accessed everywhere through a single global access point. Useful for things like configuration, loggers, connection pools, and caches. Easy to get wrong. Surprisingly hard to do right once reflection, serialization, and multiple class loaders enter the picture.

The Core Idea

A singleton enforces three rules:

1. The class can't be instantiated from outside (private constructor).
2. The class holds its own instance internally.
3. A static method hands that instance out.

```
public class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

This is the textbook version. And it's broken in any multi-threaded environment, which is most real systems.

Implementation Approaches

1. Eager Initialization

Create the instance when the class loads. Simplest approach that's actually thread-safe.

```
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();

    private Singleton() {}

    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

Pros: thread-safe through JVM class loading guarantees, dead simple.

Cons: the instance is created even if `getInstance()` is never called. Wasteful for heavy objects you might not need.

2. Lazy Initialization (Not Thread-Safe)

Defer creation until the first call. Don't ship this in multi-threaded code.

```
public class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Two threads can both pass the null check and both call `new`, ending up with two instances. Treat this as the broken baseline you're improving on.

3. Synchronized Method

Cheap fix, expensive at runtime.

```

public class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

```

Pros: thread-safe.

Cons: every call grabs a lock, even after the instance exists. Slow under contention.

4. Double-Checked Locking

Lock only on the first creation. Requires `volatile` to work correctly.

```

public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

Pros: lazy, thread-safe, no lock on the hot path.

Cons: subtle. The `volatile` keyword is non-negotiable. Without it, one thread can see a partially-constructed instance because of JVM instruction reordering.

5. Bill Pugh / Holder Idiom

Cleanest lazy singleton. Uses class loader semantics to guarantee thread safety with no synchronization.

```
public class Singleton {
    private Singleton() {}

    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

The inner `Holder` class isn't loaded until `getInstance()` is called for the first time. Class loading is thread-safe by spec. This is the recommended pattern when you can't or don't want to use an enum.

6. Enum Singleton

Joshua Bloch's recommendation in *Effective Java*. Concise, and immune to reflection and serialization attacks.

```
public enum Singleton {
    INSTANCE;

    public void doSomething() {
        // ...
    }
}

// Usage
Singleton.INSTANCE.doSomething();
```

Pros: thread-safe by JVM design, can't be broken by reflection or serialization, one line of code.

Cons: can't extend another class (enums already extend `Enum`). Initialization happens at class load, so it's eager, not lazy.

Breaking the Singleton

Standard singletons crack under three attacks. The enum singleton is immune to all of them.

Reflection

```
Constructor<Singleton> c = Singleton.class.getDeclaredConstructor();
c.setAccessible(true);
Singleton fake = c.newInstance();
```

Defense: throw from the constructor if the instance already exists.

```
private Singleton() {
    if (instance != null) {
        throw new IllegalStateException("instance already exists");
    }
}
```

Serialization

Serializing a singleton and deserializing it produces a fresh instance.

Defense: implement `readResolve()` to return the existing instance after deserialization.

```
private Object readResolve() {
    return getInstance();
}
```

Cloning

If the singleton implements `Cloneable`, calling `clone()` produces a new object.

Defense: don't implement `Cloneable`. If forced to, override `clone()`:

```
@Override
protected Object clone() throws CloneNotSupportedException {
    return getInstance();
}
```

When to Use Singleton (and When to Skip It)

Use it for:

- A logger that wraps an expensive resource.
- Configuration loaded once at startup.
- A cache shared across the app.

Skip it when:

- You're using a DI framework (Spring, Guice, Dagger). They manage singleton lifecycle for you through `@Singleton` annotations or bean scopes. Building your own on top means two instances and confusion about which one is "the" one.
 - The "single instance" requirement is more about convenience than correctness. Global state is harder to test and harder to reason about than passing the dependency in.
 - The class has no state. A `final` class with static methods and a private constructor is cleaner.
-

Quick Reference

Approach	Thread-safe?	Lazy?	Reflection-safe?	Verdict
Eager	Yes	No	No	Fine for cheap objects
Lazy (basic)	No	Yes	No	Broken
Synchronized method	Yes	Yes	No	Slow
Double-checked locking	Yes	Yes	No	Works, but verbose
Bill Pugh / Holder	Yes	Yes	No	Best non-enum option
Enum	Yes	No	Yes	Best when you can use it

Common Gotchas

- A singleton in one class loader isn't the same instance as one in another. Web containers and OSGi runtimes can break the "one per JVM" assumption.
- Static fields hold references for the JVM's lifetime. A singleton that holds large objects is effectively a memory leak.
- Don't use the singleton itself as a lock target if external code can also lock on it. Use a private lock object instead.
- If `getInstance()` recurses back into the same class during initialization, you can deadlock the class loader. Nasty to debug.
- Spring's default bean scope is singleton. If you also write your own `getInstance()`, you'll end up with two instances and no clear story about which one wins.
- A `Singleton` that holds a `Connection`, `Thread`, or `ExecutorService` will keep them alive forever. Make sure that's what you want.